P0332r0 : Relaxed Incomplete Multidimensional Array Type Declaration

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0332r0
Date:	2016-05-27
Reply-to:	hcedwar@sandia.gov, balelbach@lbl.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Bryce Lelbach
Contact:	balelbach@lbl.gov
Author:	Christian Trott
Contact:	crtrott@sandia.gov
Author:	Mauro Bianco
Contact:	mbianco@cscs.ch
Author:	Robin Maffeo
Contact:	Robin.Maffeo@amd.com
Author:	Ben Sander
Contact:	ben.sander@amd.com
Audience:	Library Evolution Working Group (LEWG)
URL:	https://github.com/kokkos/array_ref/blob/master/proposals/P0332.rst

Revision History	
P0009r0	Original multidimensional array reference paper with motivation, specification, and examples.
P0009r1	Revised with renaming from `view` to `array_ref` and allow unbounded rank through variadic arguments.
P0332r0 *(current)*	Relaxed array declaration moved from P0009.
References	
P0009r2	Multidimensional array reference specification
P0331	Multidimensional array reference motivation and examples

Motivation

The dimensions of multidimensional array reference class `array_ref` (P0009r2) are declared through either the data type or array properties template arguments.

```
template< typename DataType , typename Properties... >
struct array_ref ;
```

`DataType` may be an array type (8.3.4.p3). For example, `array_ref<double[ ][3][8]>` declares a rank-three array with a dynamic extent in dimension coordinate zero. This compact and intutive syntax is currently limited to a declaring a single dynamic extent in the array's leading coordinate. Removing this limitation allows declaration of an arbitrary number of dynamic extents in any coordinate.

Proposal

The current array type declarator constraints are defined in in **n4567 8.3.4.p3** as follows.

> *When several "array of" specifications are adjacent, a multidimensional array type is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted. In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (8.3.5). An array bound may also be omitted when the declarator is followed by an initializer (8.5). In this case the bound is calculated from the number of initial elements (say, N) supplied (8.5.1), and the type of the identifier of D is "array of N T". Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of*

a static data member of a class.

The preferred syntax requires a relaxation of array type declarator constraints defined in **n4567 8.3.4.p3** exclusively for an incomplete object type. The following wording change is proposed.

> *When several "array of" specifications are adjacent, a multidimensional array type is created. The first of the constant expressions that specify the bounds of the array may be omitted in some cases in the declaration of a function parameter (8.3.5). The first array bound may also be omitted when the declarator is followed by an initializer (8.5). In this case the first bound is calculated from the number of initial elements (say, N) supplied (8.5.1), and the type of the identifier of D is "array of N T". Furthermore, if there is a preceding declaration of the entity in the same scope in which the first bound was specified, an omitted array first bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.* **In declarations in which an incomplete object type is allowed any of the constant expressions that specify the bounds of the array may be ommitted.**

This minor language specification change has been implemented with a trivial (one line) patch to Clang and was permissible in gcc prior to version 5.